

TASK 01 — REST API Fundamentals & HTTP Standards

Step 1 — Study REST Fundamentals

1.REST

REST stands for **Representational State Transfer**.

It is an architectural style used to design APIs that allow applications to communicate over HTTP.

Simple example

Suppose we have a **User Service**.

The client wants to get user details.

GET /users/1

The server returns:

```
{
  "id": 1,
  "name": "Ravi",
  "email": "ravi@gmail.com"
}
```

The client requested user 1, and the server returned its details.

REST is not a programming language or framework. It is a way of designing APIs.

2. REST vs SOAP

Both REST and SOAP are used for communication between applications.

REST	SOAP(Simple object Access Protocol)
Architectural style	Protocol
Commonly uses HTTP	Uses XML-based messaging
Usually uses JSON	Commonly uses XML
Lightweight and simple	More structured and complex
Easy to use with web and mobile apps	Common in enterprise systems
Example: GET /users/1	Example: XML SOAP request

Simple understanding:

- **REST:** "Give me the user details."
- **SOAP:** "Here is a formally structured XML message requesting the user details."

3. Resource-Based Architecture

In REST, everything is treated as a **resource**.

Examples:

- User
- Product
- Order
- Payment
- Ride

Each resource has a URL.

/users

/products

/orders

/payments

/rides

Example : GET /users

Means: Give me all users.

GET /users/10

Means: Give me user with ID 10.

GET /orders/25

Means: Give me order with ID 25.

GET /users

POST /users

The HTTP method already tells us the action.

4. Stateless Communication

Stateless means the server does not remember the previous request.

Every request must contain all the information needed to process it.

Example

GET /users/10

Authorization: Bearer token123

The server can process this request using the information provided.

Simple example

Imagine going to a restaurant every day.

You tell the waiter:

“I want dosa.”

The waiter should understand the request without remembering your previous order.

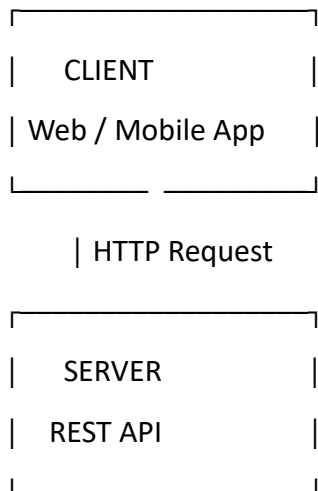
That is the basic idea of **stateless communication**.

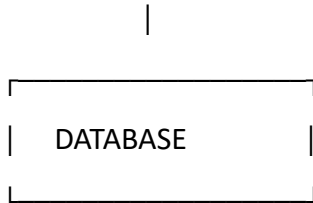
Benefits:

- Easier to scale
- Easier to maintain
- Requests can be handled independently

5. Client-Server Architecture

REST separates the **client** and **server**.





Example

The client asks: GET /products

The server:

1. Receives the request.
2. Gets products from the database.
3. Sends the response.

The client does not need to know how the database works.

Client = User interface

Server = Business logic + data

6. JSON Request and Response

JSON is a common format used to exchange data between client and server.

JSON request

Suppose we want to create a user.

POST /users

Content-Type: application/json

Request body:

```
{
  "name": "Ravi",
  "email": "ravi@gmail.com"
}
```

JSON response

```
{
  "id": 1,
  "name": "Ravi",
  "email": "ravi@gmail.com"
}
```

```
}
```

Simple understanding:

- **Request:** Client sends data.
- **Response:** Server sends data back.

Step 2 — Study HTTP Methods

HTTP methods tell the server **what operation the client wants to perform**.

Method	Meaning	Example
GET	Read data	Get users
POST	Create data	Create a user
PUT	Complete update	Replace user details
PATCH	Partial update	Change only email
DELETE	Delete data	Delete a user

1. GET → Read

Used to retrieve data.

```
GET /users
```

Meaning: Get all users.

```
GET /users/1
```

Meaning: Get user with ID 1.

Example response

```
{  
  "id": 1,  
  "name": "Ravi"  
}
```

GET does not normally create or update data.

2. POST → Create

Used to create a new resource.

POST /users

Request body:

```
{  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

Example response:

```
{  
  "id": 1,  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

3. PUT → Complete Update

Used to **replace the complete resource**.

Suppose the existing user is:

```
{  
  "id": 1,  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

Request:

PUT /users/1

Body:

```
{  
  "name": "Ravi Kumar",  
  "email": "ravikumar@gmail.com"
```

```
}
```

Replace the user details with the new complete details.

PUT is generally used when you want to send the complete updated resource.

4. PATCH → Partial Update

Used to update **only some fields**.

Suppose we only want to change the email.

```
PATCH /users/1
```

Body:

```
{  
  "email": "newemail@gmail.com"  
}
```

Change only the email. Keep the other details unchanged.

PUT vs PATCH

PUT:

```
{  
  "name": "Ravi Kumar",  
  "email": "newemail@gmail.com"  
}
```

PATCH:

```
{  
  "email": "newemail@gmail.com"  
}
```

Simple difference:

- **PUT = Complete update**
- **PATCH = Partial update**

5. DELETE → Delete

Used to remove a resource.

```
DELETE /users/1
```

Delete user with ID 1.

Example response:

204 No Content

This means the deletion was successful, and there is no response body.

Step 3: Study HTTP Status Codes

HTTP status codes are **three-digit numbers returned by the server** to tell the client what happened after sending a request.

Status Code	Name	Simple Explanation	Example
200	OK	Request completed successfully.	Get product details
201	Created	A new resource was successfully created.	Add a new product
204	No Content	Request succeeded, but there is no response body.	Delete a product
400	Bad Request	Client sent invalid data.	Product name is empty
401	Unauthorized	Authentication is required or failed.	Missing/invalid login token
403	Forbidden	User is authenticated but does not have permission.	Customer tries to delete a product
404	Not Found	Requested resource does not exist.	Product ID 10 not found
409	Conflict	Request conflicts with existing data or state.	Product with same SKU already exists
500	Internal Server Error	Unexpected error occurred on the server.	Database failure or unhandled exception

Step 4: Create Product Service

We will use **Spring Initializr** to create the project.

Open: [Spring Initializr](#)

Project Details

Field	Value
Project	Maven
Language	Java
Spring Boot	Select a stable version
Group	com.example
Artifact	product-service
Name	product-service
Packaging	Jar
Java	17 or 21

Add Dependencies

Select:

1. **Spring Web**
2. **Spring Data JPA**
3. **H2 Database**
4. **Validation**

Then click **Generate**.

Extract the downloaded ZIP file and open it in **IntelliJ IDEA** or **Eclipse**

Configure Port 8083

Open:src/main/resources/application.properties

Add:

spring.application.name=product-service

server.port=8083

Explanation

server.port=8083

This tells Spring Boot:

"Run the Product Service on port 8083."

So your service URL will be: `http://localhost:8083`

We use 8083

Because different microservices should run on different ports.

User Service → 8081

Order Service → 8082

Product Service → 8083

This allows all services to run simultaneously.

Step 4.4: Create Product Entity

Create a package:

`entity`

Inside it, create:

`Product.java`

Step 5 — Create Product Entity

An **Entity** is a Java class that represents a product in your application.

Example, if you sell mobile phones, one product might look like this:

```
{
  "id": 1,
  "name": "Samsung Galaxy S24",
  "description": "A smartphone with a high-quality camera",
  "price": 45000,
  "category": "Electronics",
  "stock": 10,
  "status": "ACTIVE",
  "createdAt": "2026-09-07T10:00:00",
  "updatedAt": "2026-09-07T10:00:00"
}
```

Example: Product.java

```

package com.example.productservice.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String description;
    private Double price;
    private String category;
    private Integer stock;
    private String status;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}

```

Step 6 — Create CRUD APIs

CRUD means:

Letter Meaning What it does

- | | | |
|----------|--------|------------------------|
| C | Create | Add a new product |
| R | Read | View products |
| U | Update | Change product details |
| D | Delete | Remove a product |

1. POST — Create a Product

POST /api/v1/products

It creates a new product.

Example request

```
{  
  "name": "Laptop",  
  "description": "HP Laptop",  
  "price": 55000,  
  "category": "Electronics",  
  "stock": 5  
}
```

Expected response

201 Created

```
{  
  "id": 1,  
  "name": "Laptop",  
  "description": "HP Laptop",  
  "price": 55000,  
  "category": "Electronics",  
  "stock": 5,  
  "status": "ACTIVE"  
}
```

"Please add this product to the database."

2. GET — Get All Products

GET /api/v1/products

It returns **all products**.

Example response

```
[  
  {  
    "id": 1,  
    "name": "Laptop",
```

```
"price": 55000
},
{
  "id": 2,
  "name": "Mobile",
  "price": 25000
}
]
```

Expected status

200 OK

"Show me all the products."

3. GET — Get Product by ID

GET /api/v1/products/1

It returns **one specific product**.

Here, 1 is the product ID.

Example response

```
{
  "id": 1,
  "name": "Laptop",
  "price": 55000,
  "stock": 5
}
```

If product does not exist

404 Not Found

"Show me product number 1."

4. PUT — Update the Entire Product

PUT /api/v1/products/1

It updates the **complete product information**.

Example request

```
{  
  "name": "Dell Laptop",  
  "description": "Updated laptop",  
  "price": 60000,  
  "category": "Electronics",  
  "stock": 8,  
  "status": "ACTIVE"  
}
```

Expected response

200 OK
"Replace the product details with these new details."

PUT vs PATCH

PUT

Updates the complete resource

Usually send all required fields

Example: Replace entire product

PATCH

Updates only selected fields

Send only fields that need changes

Example: Change only price

5. PATCH — Update Part of a Product

PATCH /api/v1/products/1

It updates **only the fields you want to change**.

Example request

```
{  
  "price": 58000,  
  "stock": 12  
}
```

Before:

Price = 55000

Stock = 5

PATCH Request:

Price = 58000

Stock = 12

After:

Price = 58000

Stock = 12

The other fields remain unchanged.

Expected response

200 OK

"Change only these product details."

6. DELETE — Delete a Product

DELETE /api/v1/products/1

It removes the product with ID 1.

Expected response

204 No Content

Usually, there is **no response body**.

Simple meaning:

"Delete product number 1."

Step 7 — Return Correct HTTP Status Codes

Because each operation has a different meaning.

Example:

POST → Product created

GET → Product retrieved

DELETE → Product deleted

So the API should return the status code that correctly describes what happened.

Correct status codes

Operation	Status Code	Meaning
Create product	201 Created	Product was successfully created
Get all products	200 OK	Products retrieved successfully
Get product by ID	200 OK	Product retrieved successfully
Update product	200 OK	Product updated successfully
Delete product	204 No Content	Product deleted successfully
Product not found	404 Not Found	Product does not exist
Invalid request	400 Bad Request	Request data is invalid
Duplicate product	409 Conflict	Request conflicts with existing data
Server error	500 Internal Server Error	Unexpected server problem

Example

Wrong:

DELETE /api/v1/products/1

200 OK

Correct:

DELETE /api/v1/products/1

204 No Content

Simple idea:

The status code tells the client what happened.

Step 8 — Test Every Endpoint in Postman

We must check whether your APIs are working correctly.

Postman is a tool used to send HTTP requests to your Spring Boot application and view the responses.

Where do you do this?

1. Open **IntelliJ IDEA / Eclipse**.
2. Run your Product Service.
3. Make sure the application is running on **port 8083**.
4. Open **Postman**.

5. Send requests to your APIs.

1. Test Successful Request

Get All Products

Method: GET

URL: http://localhost:8083/products

Expected Response:

200 OK

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 50000
  },
  {
    "id": 2,
    "name": "Mouse",
    "price": 1000
  }
]
```

Whether the API returns the products successfully.

2. Test Invalid ID

Example

Method: GET

URL: http://localhost:8083/products/abc

Expected Response:

400 Bad Request

If your API uses a different ID format, test an ID that is invalid for your implementation.

3. Test Missing Fields

Create Product Without Name

Method: POST

URL: http://localhost:8083/products

Body → raw → JSON:

```
{  
  "price": 50000,  
  "description": "Dell Laptop"  
}
```

Expected Response:

400 Bad Request

Whether your validation is working correctly.

4. Test Update

Update Product

Method: PUT

URL: http://localhost:8083/products/1

Body:

```
{  
  "name": "Gaming Laptop",  
  "price": 75000,  
  "description": "Updated laptop"  
}
```

Expected Response:

200 OK

```
{  
  "id": 1,  
  "name": "Gaming Laptop",  
  "price": 75000,
```

```
"description": "Updated laptop"
}
```

Whether the existing product is updated correctly.

5. Test Delete

Delete Product

Method: DELETE

URL: http://localhost:8083/products/1

Expected Response:

204 No Content

Whether the product is deleted successfully.

6. Test Non-Existing Product

Example

Method: GET

URL: http://localhost:8083/products/999

Expected Response:

404 Not Found

```
{
  "message": "Product not found"
}
```

Step 9 — Document the API

Product Service API Documentation

Base URL :http://localhost:8083

1. Get All Products

Endpoint

GET /products

Description

Retrieves all products.

Success Response

200 OK

Example Response

```
[  
  {  
    "id": 1,  
    "name": "Laptop",  
    "price": 50000,  
    "description": "Dell Laptop"  
  }  
]
```

2. Get Product By ID

Endpoint

GET /products/{id}

Description

Retrieves a product using its ID.

Success Response

200 OK

Not Found Response

404 Not Found

3. Create Product

Endpoint

POST /products

Description

Creates a new product.

Request Body

```
{  
  "name": "Laptop",  
  "price": 50000,
```

```
"description": "Dell Laptop"
}
```

Success Response

201 Created

Invalid Request

400 Bad Request

4. Update Product

Endpoint

PUT /products/{id}

Description

Updates an existing product.

Request Body

```
{
  "name": "Gaming Laptop",
  "price": 75000,
  "description": "Updated laptop"
}
```

Success Response

200 OK

Not Found Response

404 Not Found

5. Delete Product

Endpoint

DELETE /products/{id}